CORE JAVA

Short Notes • 10 Topic Sections

Undergraduate Computer Science Study Material

Prepared for uploading to BCS Pedia.

1. Introduction to Java

Meaning

Java is a high-level, class-based programming language. Java source code is compiled into bytecode, which can run on systems with a compatible Java Virtual Machine (JVM).

Features

Important features include object-oriented programming, portability, security features, robustness, multithreading, automatic memory management, and a large standard library.

Uses

Java is used for enterprise applications, server-side systems, desktop software, network applications, and many learning projects.

First program

The main method is the usual entry point for a simple Java application.

Example

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Execution

Save as Hello.java, compile with **javac Hello.java**, then run with **java Hello**. The compiler creates bytecode in a .class file.

2. JDK, JRE and JVM

JVM

The Java Virtual Machine loads, verifies, and executes Java bytecode. It manages runtime memory and may use a Just-In-Time compiler for performance.

JRE

The Java Runtime Environment traditionally includes the JVM and libraries needed to run Java applications.

JDK

The Java Development Kit contains tools for creating Java programs, such as the compiler, launcher, debugger, and runtime components.

Comparison

JVM executes bytecode; JRE supplies a runtime environment; JDK supplies development tools. A developer normally installs a JDK.

Compilation flow

Source code (.java) → Java compiler → bytecode (.class) → JVM → program execution.

3. Variables, Types and Operators

Variables

A variable is a named location used to store a value. Java requires a variable to be declared with a type before use.

Primitive types

Java has eight primitive types: byte, short, int, long, float, double, char, and boolean. The first six represent numeric values, char represents a UTF-16 code unit, and boolean stores true or false.

Reference types

Classes, arrays, interfaces, and strings are reference types. A reference variable refers to an object or array.

Operators

Arithmetic: +, -, *, /, %. Relational: ==, !=, <, >, <=, >=. Logical: &&, ||, !. Assignment: =, +=, -=.

Example

```
int a = 12;
int b = 5;
System.out.println(a + b); // 17
System.out.println(a % b); // 2
```

4. Control Statements and Arrays

Decision making

if, if-else, else-if, and switch allow a program to choose which instructions to execute.

Loops

for is useful for counted repetition. while checks its condition before each iteration. do-while executes the body at least once.

Jump statements

break exits a loop or switch; continue skips to the next loop iteration; return exits a method.

Arrays

An array stores a fixed-size sequence of values of the same type. Array indexes begin at zero, and the length property gives the number of elements.

Example

```
int[] marks = {70, 82, 91};
for (int i = 0; i < marks.length; i++) {
    System.out.println(marks[i]);
}
```

5. Classes, Objects and Encapsulation

Class

A class is a blueprint that defines fields and methods. It may also contain constructors and nested types.

Object

An object is an instance of a class. It has state, behavior, and identity. The new operator commonly creates objects.

Fields and methods

Fields store data. Methods define behavior. Instance members belong to objects; static members belong to the class.

Encapsulation

Encapsulation combines data and methods and controls access to internal state. A common practice is to keep fields private and provide public methods that validate changes.

Example

```
class Student {
    String name;
    void display() { System.out.println(name); }
}
Student s = new Student();
s.name = "Asha";
s.display();
```

6. Constructors, this and static

Constructors

A constructor initializes a new object. It has the same name as the class and has no return type. Constructors can be overloaded.

Default constructor

If no constructor is declared, Java provides a no-argument default constructor. If a constructor is declared, that automatic default is not supplied.

this keyword

this refers to the current object. It can distinguish a field from a parameter or call another constructor using this(...).

static keyword

A static field is shared by the class. A static method can be called using the class name and cannot directly use instance members without an object.

Example

```
class Counter {
    static int total = 0;
    Counter() { total++; }
}
new Counter();
new Counter();
System.out.println(Counter.total); // 2
```

7. Inheritance and Polymorphism

Inheritance

Inheritance allows a subclass to reuse and specialize accessible members of a superclass. Java classes use extends; classes can implement multiple interfaces.

Common forms

Single, multilevel, and hierarchical inheritance are common teaching categories. Java does not allow a class to extend multiple classes.

Overloading

Overloading means using the same method name with different parameter lists. The compiler chooses the suitable method based on arguments.

Overriding

Overriding occurs when a subclass provides a compatible implementation of an inherited instance method.

Polymorphism

Polymorphism allows a common superclass or interface reference to represent different object types, with overridden behavior selected at runtime.

8. Abstraction, Interfaces and Packages

Abstraction

Abstraction shows essential operations and hides implementation details. An abstract class cannot be instantiated directly and may contain abstract and concrete methods.

Interface

An interface describes a contract that implementing classes agree to follow. Modern Java interfaces can include abstract, default, static, and private methods.

Access modifiers

public is broadly accessible; private is limited to the declaring class; protected has package and subclass access rules; no modifier gives package-private access.

Packages

A package groups related classes and interfaces, organizes code, and helps control access. import lets code refer to types by simple name.

Design tip

Use interfaces to define capabilities, abstract classes to share common implementation, and encapsulation to protect valid object state.

9. Exception Handling and Strings

Exceptions

An exception represents an abnormal condition. Checked exceptions must be caught or declared; unchecked exceptions include many runtime errors.

Keywords

try encloses risky code; catch handles a matching exception; finally is commonly used for cleanup; throw explicitly throws an exception; throws declares possible exceptions.

Strings

String represents text and is immutable. Methods such as length(), charAt(), substring(), equals(), and contains() are commonly used.

Compare strings

Use equals() to compare text contents. The == operator compares reference identity and is usually not suitable for checking whether two strings contain the same text.

Example

```
try {
    int n = Integer.parseInt("25");
    System.out.println(n);
} catch (NumberFormatException e) {
    System.out.println("Invalid number");
}
```

10. Collections, Threads and Revision

Collections

List stores an ordered sequence and permits duplicates; Set models unique elements; Map stores key-value pairs. ArrayList is a resizable list and HashMap stores key-value mappings.

Threads

A thread is a path of execution within a process. A thread can be created by extending Thread or implementing Runnable. Shared mutable data needs careful coordination.

Key terms

JDK is used for development, JVM runs bytecode, a class is a blueprint, an object is an instance, inheritance supports reuse, and polymorphism supports flexible behavior.

Revision questions

1. Explain JDK, JRE and JVM. 2. Differentiate overloading and overriding. 3. Explain constructors and this. 4. Define encapsulation. 5. Explain exception handling. 6. Differentiate List, Set and Map.